AIMERZ.ai
Aim.Act.Achieve

NOTES

Wrapper Classes & Generics

Working with Wrapper Objects in Java

Introduction to Wrapper Classes: In Java, wrapper classes are used to convert primitive data types into objects. They "wrap" the primitive types (like int, char, etc.) in a class, enabling them to be treated as objects. Wrapper classes provide a way to use primitive data types (such as int, float, char, etc.) in contexts that require objects, such as in collections like ArrayList and HashMap.

Real-Life Analogie: Wrapper Classes as Gift Wrappers

Think of a primitive value as a gift. A wrapper class wraps this gift, making it suitable for certain occasions (like a party or presentation). The gift remains the same, but now it is better suited for the context.

Java provides a wrapper class for each of the primitive data types:

Primitive Type	Wrapper Class
byte	Byte
short	Short
int	Integer
long	Long
float	Float
double	Double
char	Character
boolean	Boolean

Let's understand, Why Wrapper Classes Are Used:

Object-Oriented Contexts: Many Java frameworks like (e.g., collections, generics) work with only objects, not primitive types. Wrapper classes enable primitives to be used in these as object-based.

```
Java
ArrayList<Integer> list = new ArrayList<>();
list.add(10); // Works because 10 is autoboxed into Integer
```

Utility Methods: Wrapper classes include methods for type conversion, parsing, and more.

Real-World Analogy: Think of wrapper classes like adapters that let an incompatible plug (primitive) fit into a socket (object).

Null Representation: Primitives can't hold null values, but wrapper classes can, which helps when working with databases or data streams where null values need to be handled.
Example in Real-Time Usage: Retrieving a nullable integer from a database:

```
Java
Integer dbValue = null; // Represents a null value from the database
```

Immutability: All wrapper class objects are immutable, meaning their value cannot be changed once created.

Let's understand what is Autoboxing and Unboxing

Autoboxing is the automatic conversion of primitive types into their corresponding wrapper class objects by the Java compiler.

Example:

```Java
int num = 10;
Integer wrappedNum = num; // Autoboxing
```

Unboxing is the reverse process, where wrapper class objects are automatically converted back into their corresponding primitive types.

Example:

```Java
Integer wrappedNum = new Integer(20);
int num = wrappedNum; // Unboxing
```

Let's see the Wrapper Class Constructors (Pre-Java 9)

Prior to Java 9, each wrapper class had constructors to create an object from a primitive or a string:

a. Integer(int value)
b Integer(String s)

Before Java 9 (Using Constructors):

```Java
Integer obj1 = new Integer(10);   // From primitive
Integer obj2 = new Integer("20"); // From string
System.out.println(obj1 + ", " + obj2); // Output: 10, 20
```

However, starting from Java 9, these constructors were deprecated in favor of the valueOf() method for object creation.

After Java 9 (Using valueOf()):

```Java
Integer obj1 = Integer.valueOf(10);   // From primitive
Integer obj2 = Integer.valueOf("20"); // From string
System.out.println(obj1 + ", " + obj2); // Output: 10, 20
```

So we should remember that: The constructor method (e.g., new Integer()) is deprecated after Java 9. Always use valueOf() for creating wrapper objects.

Let's dive deep in to the, Common Methods in Wrapper Classes

valueOf() Method: Converts a primitive or string to a wrapper object.

```Java
Integer obj = Integer.valueOf(100); // From primitive
Integer objStr = Integer.valueOf("100"); // From string
```

parse<Type>() Method: The parse<Type>() methods are static methods in wrapper classes that convert a String to a corresponding primitive value.

```Java
int num = Integer.parseInt("123");
float fnum = Float.parseFloat("45.6");
```

xxxValue() Method: The xxxValue() methods are used to convert wrapper objects back into primitive values.

```Java
Integer iObj = Integer.valueOf(10);

int i = iObj.intValue();  // Convert to primitive int

Double dObj = Double.valueOf(10.5);
```

```
double d = dObj.doubleValue();  // Convert to primitive double
```

toString() Method: Converts a wrapper object into a string.

```Java
Integer iObj = Integer.valueOf(100);

String str = iObj.toString();  // "100"
```

compareTo() Method: The compareTo() method compares two wrapper objects and returns:

0 if both are equal

Positive number if the calling object is larger

Negative number if the calling object is smaller

Integer i1 = Integer.valueOf(15);

Integer i2 = Integer.valueOf(20);

int result = i1.compareTo(i2); // result will be -1 because 15 < 20

equals() Method: The equals() method compares the value of two wrapper objects for equality.

```Java
Integer i1 = Integer.valueOf(100);

Integer i2 = Integer.valueOf(100);

boolean isEqual = i1.equals(i2);  // true, as both have the
same value
```

Character-specific Methods

➔ **`isDigit(char)`**: Checks if a character is a number.

➔ **`isLetter(char)`**: Checks if a character is a letter.

Example for Character class:

```
Java
Character.isDigit(char c): Checks if the character is a digit.

Character.isLetter(char c): Checks if the character is a
letter

boolean isDigit = Character.isDigit('5');  // true

boolean isLetter = Character.isLetter('A'); // true
```

hashCode() Method: Returns a hash code for the wrapper object, which is generally the same as the primitive value's hash code.

```
Java
Integer i = Integer.valueOf(10);

int hash = i.hashCode();  // 10, since hash code for
Integer is the value itself
```

Let's see, Type Conversion with Wrapper Classes

Primitive to String: Wrapper classes provide methods like toString() or String.valueOf() to convert primitive types into strings.

```
Java
int num = 100;
String str = Integer.toString(num);   // "100"
```

String to Primitive: Wrapper classes have methods like parse<Type>() and valueOf() to convert strings into primitive types.

```Java
String str = "123";
int num = Integer.parseInt(str);   // 123
```

Let's see the, Caching in Wrapper Classes:

Java caches frequently used values in wrapper classes, especially for integers between -128 and 127, which are stored in a cache to avoid creating new objects each time. This is known as the Integer Cache.

Example:

```Java
Integer i1 = Integer.valueOf(100); // Cached
Integer i2 = Integer.valueOf(100); // Same object as i1
System.out.println(i1 == i2); // true

Integer i3 = Integer.valueOf(200); // Not cached
Integer i4 = Integer.valueOf(200); // New object
System.out.println(i3 == i4); // false
```

Introduction to Generics

Generics in Java allow the creation of classes, interfaces, and methods that can operate on any type of data (Object types). They enable type safety, code reusability, and **compile-time type checking**. Generics are a powerful feature added to Java in version 5.0, addressing the need for more flexible, yet type-safe operations on various types.

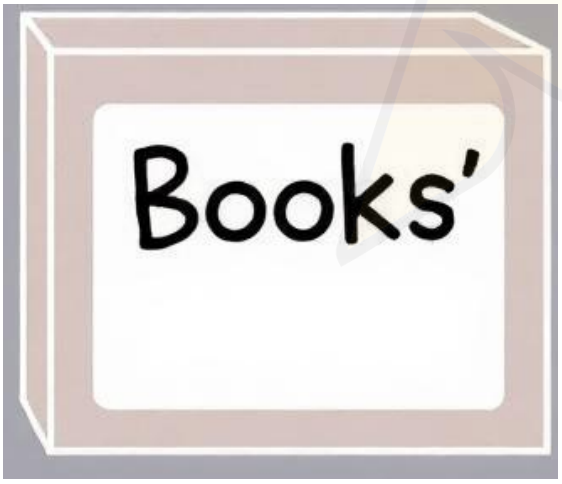
Definition: Generics provide a way to create a placeholder for a data type. Instead of specifying a concrete data type, you can use a generic type parameter.

Lets see, a. Key Benefits of Generics:

- ➔ **Type Safety**: Ensures that you use the correct type of data.
- ➔ **Code Reusability**: Write code that works with any data type.
- ➔ **Avoid Casting**: Reduces the need for explicit type casting.
- ➔ **Elimination of Casts:** Reduces the need for explicit casting when extracting elements from collections.
- ➔ **Compile-Time Checking:** Catches errors early at compile time rather than at runtime.

Real-World Analogy: Container Boxes

Imagine a labeled container box (e.g., a box for "Books" or "Toys").
Without generics: You can put anything in the box, but you don't know its type until you open it.
With generics: The label ensures only specific items (e.g., books) can go in the box.



Lets understand, What are Generic Classes?
Before generics, code reusability was often implemented using Object references, but this resulted in unsafe type casting and made the code prone to ClassCastException at runtime.

Example Without Generics:

```Java
public class Box {
    private Object item;

    public void set(Object item) {
        this.item = item;
    }

    public Object get() {
        return item;
    }

    public static void main(String[] args) {
        Box box = new Box();

        // Storing a String
        box.set("Hello, World!");

        // Retrieving and casting to String
        String message = (String) box.get();
        System.out.println(message); // Output: Hello, World!

        // Storing an Integer
        box.set(123);

        // Retrieving and casting to String (incorrect)
        String incorrectMessage = (String) box.get(); //
Throws ClassCastException
        System.out.println(incorrectMessage);
    }
}
```

Output

```Java
Hello, World!
Exception in thread "main" java.lang.ClassCastException: class
java.lang.Integer cannot be cast to class java.lang
```

In this code, because there are no type checks, a runtime error will occur when trying to cast an integer to a string.

Now we have seen the error above so how to resolve this and how generics help us in this and before getting into generic class lets see the Generic Types Syntax:

Generics are specified using angle brackets <>. Here's the basic syntax for using generics:

```Java
class ClassName<T> {

      // Code goes here

}
```

Where T represents a **generic type parameter**.

Now let's create a Generic class and solve our above problem

Generic Class

```Java
class Box<T> {
    private T item;

    public void set(T item) {
        this.item = item;
    }

    public T get() {
        return item;
    }
}
```

Usage:

```java
Box<String> stringBox = new Box<>();
stringBox.set("Hello, World!");
System.out.println(stringBox.get()); // Output: Hello, World!
```

In this example, Box can hold objects of any type, ensuring type safety and eliminating the need for type casting.

Real-World Example:

A **parcel service** only delivers specific types of packages, such as documents, clothes, or electronics. The Box<T> ensures the type consistency of items in the box.

Let's see the **Generic Methods:** If a method is designed to work with multiple types, you can define it with a generic parameter.
Example:

```java
public static <T> void printArray(T[] elements) {
    for (T element : elements) {
        System.out.println(element);
    }
}
```

Usage

```java
String[] names = {"John", "Jane", "Doe"};
Integer[] numbers = {1, 2, 3};

printArray(names); // Prints all names
```

```
printArray(numbers); // Prints all numbers
```

Real-World Example:

A **printer** that can print documents, photos, or receipts. The generic method ensures the printer can handle all these types.

Generic Interfaces: We can also define interfaces with generics.

Example:

```Java
interface Storage<T> {
    void add(T item);
    T get(int index);
}

class StringStorage implements Storage<String> {
    private List<String> items = new ArrayList<>();

    public void add(String item) {
        items.add(item);
    }

    public String get(int index) {
        return items.get(index);
    }
}
```

Usage

```Java
StringStorage storage = new StringStorage();
storage.add("Item 1");
```

```
System.out.println(storage.get(0)); // Output: Item 1
```

Real-World Example:

A **warehouse** may store only specific types of goods, such as electronics, furniture, or food items. The generic interface defines this behavior.

Let's also understand **What are Wildcard in Generics:**

Java generics also provide a special mechanism known as wildcards which allow more flexible method definitions and parameterization. The wildcard is represented by a question mark ? and is useful when you don't know the type of the generic class or interface.

Types:
Unbounded WildCards

- ➔ <?>: Unbounded wildcard (any type)

Bounded WildCards

- ➔ <? extends T>: Upper-bounded wildcard (type T or its subclasses)
- ➔ <? super T>: Lower-bounded wildcard (type T or its superclasses)

Unbounded Wildcard: Used when any type can be accepted.

```Java
// Generic Box class that can store any type of object
class Box<T> {
    private T item; // Item of generic type T

    // Constructor to initialize the Box with an item
    public Box(T item) {
        this.item = item;
    }

    // Getter method to retrieve the item
    public T getItem() {
```

```java
        return item;
    }

    // Setter method to update the item
    public void setItem(T item) {
        this.item = item;
    }
}

// Printer class that demonstrates the use of unbounded wildcard
class Printer {
    // Method that accepts a Box of any type using unbounded wildcard
(<?>)
    public void printBox(Box<?> box) {
        // Printing the contents of the Box
        System.out.println("Box contains: " + box.getItem());
    }
}

public class Main {
    public static void main(String[] args) {
        // Create a Box object to store a String
        Box<String> stringBox = new Box<>("Hello, World!");

        // Create a Box object to store an Integer
        Box<Integer> integerBox = new Box<>(42);

        // Create a Printer object
        Printer printer = new Printer();

        // Printing the contents of the String box
        printer.printBox(stringBox); // Output: Box contains: Hello,
World!

        // Printing the contents of the Integer box
        printer.printBox(integerBox); // Output: Box contains: 42
    }
}
```

Understanding above code:

Box Class:

- ➔ T is a placeholder for any type.
- ➔ Comments explain the purpose of the constructor, getter, and setter methods.

Printer Class:

- ➔ The printBox method uses <?> (unbounded wildcard) to accept Box objects of any type.
- ➔ This is useful when the method does not depend on the type stored in the box.

Main Method:

- ➔ Demonstrates how to create Box objects with different types (String and Integer).
- ➔ Shows how the Printer class can handle both types using the unbounded wildcard.

Output

```java
Box contains: Hello, World!
Box contains: 42
```

Bounded Wildcard: Limits the types that can be used with generics.

And are of two types

What are Upper-Bounded Wildcards in Java?

Upper-bounded wildcards allow you to specify that a type parameter is either a specific type or a subtype of that type. This is useful when you want to restrict the types of objects that can be passed to a method.

The syntax for an upper-bounded wildcard is:
<? extends Type>
This means the type can be Type or any class that extends Type.

Example with Explanation

```Java
import java.util.List;

public class UpperBoundExample {
    // A method that calculates the sum of a list of numbers
(Integer, Double, etc.)
    public static double calculateSum(List<? extends Number> numbers)
{
        double sum = 0.0;
        for (Number num : numbers) {
            sum += num.doubleValue(); // Convert to double for
addition
        }
        return sum;
    }

    public static void main(String[] args) {
        List<Integer> intList = List.of(10, 20, 30); // A list of
integers
        List<Double> doubleList = List.of(1.5, 2.5, 3.5); // A list
of doubles

        // Using the calculateSum method with different types
        System.out.println("Sum of integers: " +
calculateSum(intList)); // Works with Integer
        System.out.println("Sum of doubles: " +
calculateSum(doubleList)); // Works with Double
    }
}
```

How It Works

Wildcard Declaration (<? extends Number>):
The ? extends Number wildcard means the method can accept any type that is Number or a subclass of Number (e.g., Integer, Double).

Input Lists:

We create a list of Integers and a list of Double.

-Both types are subclasses of Number, so they are valid inputs for the method.

Iteration and Conversion:

The method iterates through the list and converts each element to a double using the doubleValue() method, which is available for all subclasses of Number.

Result:
The method adds up the values and returns the sum.

Why Use Upper-Bounded Wildcards?

It ensures type safety while allowing flexibility. For instance, you don't need to write separate methods for Integer and Double; you can handle both with one method using <? extends Number>.

What are lower-bounded wildcards in Java?

Lower-bounded wildcards restrict the type to a specific class or any of its superclasses using super.

Lower-bounded wildcards in Java allow you to specify that a type parameter must be of a certain type or a supertype of that type. This is useful when you want to ensure that a method can accept a wide range of types while guaranteeing compatibility with a specific base type.

The syntax for a lower-bounded wildcard is:
<? super Type>
This means the type can be Type or any of its supertypes.

Example with Explanation

```Java
import java.util.List;
import java.util.ArrayList;

public class LowerBoundExample {
     // A method to add integers to a list of numbers or their
supertypes
     public static void addNumbers(List<? super Integer> list)
{
        for (int i = 1; i ≤ 5; i++) {
```

```
            list.add(i); // Adding integers to the list
        }
    }

    public static void main(String[] args) {
         List<Number> numberList = new ArrayList<>(); // A list
of Number (supertype of Integer)
         List<Object> objectList = new ArrayList<>(); // A list
of Object (supertype of Number)

        // Add numbers to the list
        addNumbers(numberList); // Works with Number
        addNumbers(objectList); // Works with Object

        System.out.println("Number List: " + numberList);
        System.out.println("Object List: " + objectList);
    }
}
```

How It Works

Wildcard Declaration (<? super Integer>):
The ? super Integer wildcard ensures that the list can accept elements of type Integer or any of its supertypes (Number, Object).
Input Lists:
We create two lists: one of type Number and one of type Object.
Both are supertypes of Integer, so they are valid inputs for the method.
Adding Elements:
Inside the method, integers (1 to 5) are added to the list. This works because Integer is compatible with Number and Object.
Output:
After calling the method, both lists contain the integers added to them.

Why Use Lower-Bounded Wildcards?

Flexibility with Parent Types:

It allows methods to work with a broader range of types while maintaining compatibility with a specific type (like Integer in this case).

Adding Elements:

Unlike upper-bounded wildcards (<? extends Type>), lower-bounded wildcards allow adding elements to the collection because the compiler knows the minimum type requirement.

Type Erasure in Generics:

What is type erasure in Java?

Type erasure is a mechanism used by the Java compiler to implement generics. It ensures that no new classes are created for parameterized types, allowing for backward compatibility with older versions of Java that do not support generics. The compiler removes all type parameters and replaces them with their bounds or Object if they are unbounded.

Real-World Analogy: Labels on jars (like "Sugar" or "Salt") are removed during a blind taste test, but the contents remain the same.

Key Points of Type Erasure

Replacement of Type Parameters:

All type parameters in generic classes and methods are replaced with their bounds or Object.
For example, a generic class Aimerz<T> where T is unbounded will be compiled to Aimerz<Object>.

Insertion of Type Casts:

The compiler inserts necessary type casts to maintain type safety after erasure.

Bridge Methods:

To preserve polymorphism in extended generic types, the compiler may generate bridge methods.

How It Works

Compile-Time Check:

Generics are checked at compile time to ensure type safety.
If a type mismatch occurs, the compiler throws an error.
Runtime:
After compile-time, the type information is erased, and generic types are replaced with raw types.

Example with Explanation

```Java
class Aimerz<T> {
    T obj;

    Aimerz(T o) {
        obj = o;
    }

    T getob() {
        return obj;
    }
}
```

After Compilation:
The above class will be transformed into:

```Java
class Aimerz {
     // Declare a variable of type Object to store any type of
data
    Object obj;

     // Constructor: Accepts any type of object and assigns it
to 'obj'
    Aimerz(Object o) {
        obj = o; // Store the passed object
    }
```

```
    // Method: Returns the stored object
    Object getob() {
        return obj; // Return the object stored in 'obj'
    }
}
```

Why Use Type Erasure?

Backward Compatibility:
Java generics were introduced in Java 5. Type erasure ensures that older code can still run without modification.
Avoid Runtime Overhead:
Since type checks are performed at compile time, there's no additional runtime cost for handling generics.

Generic Restrictions:

Cannot Instantiate Generic Types with Primitives: Generics in Java only work with reference types (e.g., Integer, String, etc.), not primitive types (int, char, etc.).

```Java
// Invalid:

List<int> list = new ArrayList<>();


// Valid:

List<Integer> list = new ArrayList<>();
```

Cannot Create Generic Arrays: You cannot create an array of a generic type.

```
Java
// Invalid:

T[] arr = new T[10];  // Compilation error
```

Cannot Use Generics in Static Context: Generic types cannot be used in **static fields** or **methods** because static members belong to the class, not an instance of the class.

```
Java
class Test<T> {

      static T obj;  // Compilation error

}
```

Real-Time Use Cases

- **Data Processing**:
  A List<T> can store records of various types (e.g., customers, products, or orders). Generics ensure type safety when processing data.
- **API Development**:
  REST APIs often use generics to handle multiple data types in responses.
- **Custom Utility Classes**:
  A generic Pair<K, V> class can store key-value pairs for configurations or settings.

THANK YOU

AIMERZ.ai

Aim.Act.Achieve

Stay updated with us!

Vishwa Mohan

Vishwa Mohan

vishwa.mohan.singh

Vishwa Mohan